

Deep Learning-Based Sentiment and Rating Prediction on Yelp Reviews

Justin Che Mark O'Brien Jacob Lebovitz Nathan Yoon

December 2025

Click here to view the Google Colab Notebook

<https://colab.research.google.com/drive/1xS-LMWDwAgd1uF06DbcixofArfm8DpxT?usp=sharing>

1 Problem Description

Online reviews are a primary way that customers communicate their experiences with restaurants. Yelp is a review platform that hosts millions of text reviews paired with a star ratings and engagement tags from other users (“useful”, “funny”, “cool”). For a single restaurant, these reviews are usually manageable by hand, but at the scale of thousands of businesses, manual reading and processing becomes difficult and time consuming. Additionally, many reviews in businesses come without quantitative star ratings attached, so it is important to discern negative and positive text-based reviews to automate customer service response priorities. Operators need automated tools that can translate raw review text into structured sentiment and rating predictions and highlight recurring themes that drive satisfaction or dissatisfaction.

This project studies how deep learning models for text can predict Yelp star ratings directly from review text, and explores deeply the tradeoffs are between simple and sophisticated text representations. We compare three levels of representation: a bag-of-words (BoW) model using one-hot counts, dense word embeddings via pretrained GloVe vectors, and contextual embeddings from a pretrained BERT transformer. At each level, we train neural networks to predict the star rating given only the review text and evaluate performance as a function of both tokenization type and training dataset size.

Beyond pure predictive accuracy, we are interested in what these models imply for real businesses. A useful model should behave sensibly on extreme reviews, be reasonably data-efficient, and be simple enough to integrate into dashboards or alerting tools. We are curious to see how far a simple BoW baseline can go, whether GloVe embeddings offer practical gains over BoW, and whether BERT provides meaningful improvements given small labeled datasets and limited compute power of local machines.

We work with a subset of the Yelp Open Dataset. From the full database we construct a “small” sample of 1,000 reviews and a “large” sample of 26,318 reviews to make experimentation computationally tractable while still reflecting realistic variability in text and ratings. Across all models, we treat the prediction task as regression onto the continuous star rating and evaluate models using mean squared error (MSE) and mean absolute error (MAE), in order to penalize greater deviations in error prediction.

We have additionally used Yelp images data (images from reviews) to build some supplementary/additional image recognition models. The aim is to see if we can accurately detect what the image is of (eg., good, drink, etc.), and what rating the corresponding business has.

2 Approach

From a business perspective, our task is to build an algorithm that takes in as input a raw Yelp review and returns an estimate of how satisfied the customer was. In an operations dashboard, such a tool could be used to flag reviews whose predicted sentiment is much worse than the current average for the business, or detect large differences in predicted and actual star value to detect fraud reviews, or route especially negative reviews to managers for prioritized follow-up. Although Yelp already contains the true star rating, we treat the prediction problem as if the rating is unknown at inference time to mirror applications on other free-text channels like surveys or emails where no rating is attached to the text-based review.

Formally, let x denote the text of a Yelp review and $y \in \{1, 2, 3, 4, 5\}$ denote its associated star rating. We consider the supervised learning problem of approximating y as a continuous variable from vectorized text x using a neural network. Given a dataset $\{(x_i, y_i)\}_{i=1}^n$, we split observations into train, validation, and test sets. For each model, parameters are learned by minimizing MSE on the training set, with hyperparameters chosen using validation performance.

The Yelp dataset also includes binary indicators for whether a review was marked “useful”, “funny”, or “cool”. These define related multi-label classification problems, but due to time and compute constraints we concentrate on the star rating regression task. Future work using this data could involve classifying reviews and training interpretable tree-based models to give insights into what key words differentiate useful from not useful reviews, giving insights to food reviewers or yelp critiques.

Furthermore, we investigated the pictures associated with reviews in our dataset, and built a classification model to identify what type of picture was being attached to each review, giving businesses a clear sense of what dishes or scenery were capturing attention of diners.

2.1 Data and preprocessing

We operate on a preprocessed review table derived from the Yelp Open Dataset. To keep training times manageable across several models, we construct two working datasets: 1,000 randomly sampled reviews and a larger set of 26,318 reviews. Each record contains the free-text review and its associated star rating.

For each dataset size, we first split into training and test sets with an 80/20 split using a fixed random seed, then split the training portion again into train and validation sets with a 90/10 split. The same splitting procedure is reused across models to ensure comparability.

Text preprocessing is intentionally minimal. For the bag-of-words representation we use `CountVectorizer`, which lowercases the text, tokenizes on whitespace and punctuation using default rules, and limits the vocabulary to the 20,000 most frequent tokens in the training set. For the GloVe and BERT models, tokenization and truncation are handled by their respective tokenizers.

2.2 Text representations and architectures

BoW Word Count We first build a simple bag-of-words model that ignores word order and semantics. `CountVectorizer` maps each review to a d -dimensional count vector. On top of this we train a feed-forward network with two hidden layers (256 and 128 ReLU units) and a single linear output neuron. The model is trained for 5 epochs with batch size 32 using Adam and MSE loss. This serves as our baseline.

GloVe embeddings To incorporate semantic information, we use pretrained GloVe word vectors. A Keras Tokenizer is fit on the large dataset, mapping each review to a sequence of up to 200 integer token IDs. We load GloVe embeddings and construct an embedding matrix mapping each token ID to its vector. The model consists of a non-trainable embedding layer, global average pooling over the sequence, one 128 unit ReLU-activated dense layers, and a linear output layer. Training uses the same optimizer, batch size, and epoch count as the BoW model.

BERT embeddings Finally, we build a model using a pretrained BERT backbone. We use the corresponding tokenizer to obtain tokens and attention masks, feed them through the BERT encoder, and extract the hidden state of the [CLS] token of dimension 768 as a sentence-level embedding. BERT weights are frozen; we train only a regression head consisting of two dense layers of 128 and 64 ReLU units with dropout, and a linear output. Because BERT forward passes are expensive, we restrict these experiments to subsamples of size 200, 500, 1,000, and 2,500 reviews.

2.3 Training and evaluation

For each representation and dataset size, we split the data into train/validation/test, fit only on the training data, train for 5 epochs while monitoring validation loss, and then evaluate on the held-out test set, recording MSE and MAE. We intentionally avoid heavy hyperparameter tuning to mirror what a users might build in a short project.

3 Results

3.1 BoW and GloVe across dataset sizes

On the 1,000-review dataset, the BoW model already outperforms the GloVe model, with MSE ≈ 2.77 and MAE ≈ 1.42 versus MSE ≈ 5.98 and MAE ≈ 2.13 for GloVe. As we move to the larger dataset and increase training size from 25% to 100%, both models improve, but the BoW network consistently dominates and continues to benefit from additional data, while GloVe appears to plateau around MSE ≈ 0.9 .

Table 1 summarizes representative performance when training on the full 26,318-review dataset for BoW and GloVe, and on 2,500 reviews for BERT (the largest subsample we used with the transformer).

Model	Train size	Test MSE	Test MAE
BoW	26,318	0.58	0.60
GloVe	26,318	0.89	0.74
BERT	2,500	0.89	0.78

Table 1: Representative test performance across models.

For a fixed amount of data (26k reviews), the simple BoW DNN achieves the lowest error. The GloVe-based model underperforms despite access to pretrained semantic information. We expect this is because sentence sentiment is determined by a few words and not overall semantic text understanding.

3.2 BERT and data efficiency

The BERT model is trained on much smaller subsamples, treating the nontrainable transformer as a feature extractor. Table 2 shows performance as a function of training size.

Train size	Test MSE	Test MAE
200	1.05	0.75
500	0.65	0.61
1,000	0.89	0.77
2,500	0.89	0.78

Table 2: BERT (frozen) performance as training size increases. Values are rounded.

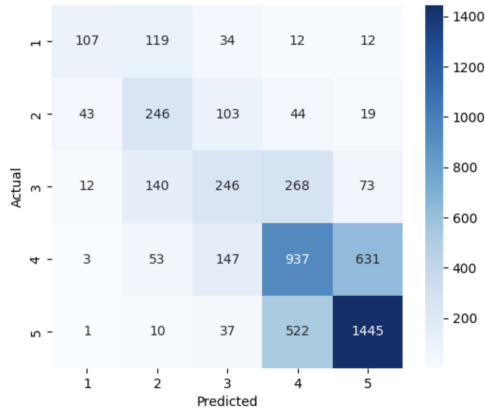
Even with as few as 200–500 labeled reviews, BERT embeddings achieve MSE and MAE in the same range as the fully trained GloVe model on tens of thousands of reviews. This highlights the strong bias of large pretrained language models, yet ability to get decent results on small datasets. The trend of error is not perfectly decreasing in sample size, reflecting some variance due to limited tuning and random subsampling, but the overall range is stable.

Relative to the BoW model trained on 26,000 reviews, the best BERT configurations are slightly worse in MSE and MAE but use an order of magnitude fewer labeled points. In settings where manual labeling is expensive or historical data is limited, this data efficiency can be more important than minimal improved accuracy.

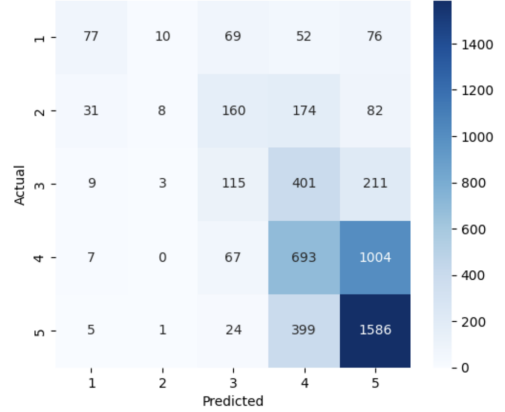
3.3 Confusion Matrices and Model Comparison

After running each model, we examined a confusion matrix of the actual vs predicted ratings. This was done using a different model where we predicted integer rating values using a softmax output layer instead of our linear continuous output layer mentioned before. This allowed us to see where the errors come from in each modeling technique, allowing businesses to chose a strategy that works for their unique needs.

BoW vs GloVe embeddings We verified through the confusion matrices that a large portion of the error in both models came from predicting a 4 for a 5, and vice versa. However, the GloVe model predicted much more 5’s for ratings that were a 4 as compared to the BoW model. Furthermore, over all review categories 1, 2, 3, 4, 5, the GloVe model overestimates the review score, while the BoW model has a more even distribution of over and under estimating the review score.



(a) BoW Confusion Matrix



(b) GloVe Confusion Matrix

Figure 1: Confusion matrices for Yelp rating models trained on 100% of data

3.4 Supplementary Image-Based Prediction

In addition to the primary text-based sentiment and rating models, we conducted a supplementary analysis using images from the Yelp dataset. Two objectives were explored: (1) predicting business star ratings based solely on images associated with reviews, and (2) classifying each image into one of several semantic categories (e.g., food, drink, menu, interior, exterior).

Rating Prediction: We trained a convolutional neural network (CNN) on grayscale 48×48 image representations (we used a separate python script to reduce the resolution and greyscale each image, and applied data augmentation (e.g., rotating images) to allow the models to generalize better). At the photo level, the model achieved a test MAE of 0.52 and RMSE of 0.69, with some individual predictions deviating by over 2 stars. Aggregating predictions per business yielded a business level MAE of 0.57 and RMSE of 0.77. These results indicate a modest but meaningful correlation between visual features and user ratings. This is likely because inferring ratings from an image is hard. e.g., a photo of the outside of a restaurant alone does not easily infer the rating (even for people).

Image Classification: A second CNN was trained to classify images into five categories provided in the Yelp dataset. Despite class imbalance, the model achieved a per-photo accuracy of 65.9%, with particularly strong performance on common categories like **food**. However, it struggled with rare classes like **menu**. At the business level (using majority vote across photos), accuracy improved to 67.15%, suggesting robustness through aggregation.

Interpretation: While not as predictive as text-based models, the image-based pipelines provide complementary insights. High-confidence misclassifications often arose from ambiguous images and due to the low resolution images, highlighting the importance of image quality and contextual cues. Overall, these results support the notion that visual media carries latent signals relevant to business reputation and thematic content.

4 Lessons Learned (Discussion/Conclusion)

Primary Models (BoW, GloVe, BERT) From a business standpoint, the main question is which model one would actually use to rate text-based reviews and identify fraud reviews. With abundant historical data and modest compute, the BoW model is compelling due to its simplicity, fast run time, and accurate test MSE.

In low-data or sparse settings, BERT embeddings are more attractive. Our results show that a few hundred labeled reviews suffice to reach performance comparable to much larger BoW or GloVe setups. For a new restaurant category or a private feedback channel with limited data, this approach can be a practical compromise between accuracy and labeling cost.

Our work also has limitations. We only used a small fraction of the full Yelp dataset, did not fine-tune BERT, and did not fully develop the interpretability pipeline proposed initially. We ran an OCT on our reviews to try to gain interpretable clusters that we could predict on. However, when understanding the resulting tree, we realized that a large majority of our data ended up in one cluster that was not interpretable, and therefore we did not continue with this branch of the project. Extending the experiments to larger datasets, modeling star ratings with auxiliary labels like “useful” or “cool”, and pairing embeddings with interpretable clusters for theme discovery would be useful next steps.

Overall, the project shows that relatively simple deep learning models can provide useful, quantitative predictions of Yelp star ratings directly from review text.

Supplementary Image models The image models were useful to improve our skills with CNN and inference from images, and examining across different applications (predicting rating vs. predicting what image is of) shows how the use case is extremely important (predicting ratings was low accuracy, predicting what the image was of was high). This shows that applying the BERT, GloVe and BoW models are more relevant for predicting ratings, but the CNN is highly relevant when identifying what images are of. A key part of deep learning is being aware of what approaches to use and when, and how to tailor approaches holistically to maximise the gain for any given scenario. Having a comprehensive understanding of the key benefits and costs of each model and approach is key to being able to apply models successfully, and to derive real world useful applications.

References

- [1] Yelp Inc. (2025). Yelp Open Dataset. <https://business.yelp.com/data/resources/open-dataset/>. Accessed 2025-11-20.

Note: Dataset originally sourced from the Yelp Open Dataset, with manual preprocessing and formatting applied to images for compatibility with the prediction pipelines and tractable image resolution (compute power limited to A100’s).